Documentation technique

Thomas Bruston

Titre Professionnel DWWM — Niveau 5
Organisme de formation Studi

Session juin-juillet 2026

1. Réflexions initiales technologiques

Contexte et contraintes

Le projet Vite & Gourmand est une application web fullstack développée sans framework, conformément aux exigences du titre professionnel DWWM.

Choix de la stack technique

PHP 8.3 orienté objet a été retenu pour le back-end. L'absence de framework impose une architecture MVC construite de zéro.

MySQL 8.0 a été choisi pour la base de données relationnelle.

MongoDB 7.0 a été intégré pour les statistiques administrateur. Les données d'analyse (commandes terminées, chiffre d'affaires par menu) n'ayant pas de contraintes relationnelles.

Docker et Docker Compose ont été adoptés pour garantir la reproductibilité de l'environnement entre le développement et la production.

PHPMailer est l'unique dépendance Composer du projet. Elle gère l'envoi des emails transactionnels (confirmation de commande, bienvenue, réinitialisation de mot de passe, retour matériel).

2. Configuration de l'environnement de travail

Outils utilisés

Visual Studio Code a été utilisé comme éditeur principal, avec les extensions suivantes : Live Server, HTML CSS Support pour l'autocomplétion, CodeRunner pour l'exécution rapide de scripts, et IntelliSense for CSS class names in HTML pour la complétion des classes CSS.

Git avec Git Flow a été mis en place pour le versionnage. La branche main contient le code stable de production, la branche develop centralise les développements en cours.

Trello a été utilisé pour la gestion des tâches sous forme de tableau Kanban.

Figma a été utilisé pour la conception des wireframes et mockups desktop et mobile.

Prérequis techniques

- Docker Desktop (version 4.x ou supérieure)
- Git
- Un navigateur moderne (Chrome, Firefox, Edge)

Structure du projet

```
/
├── public/        ← point d'entrée unique (index.php)
├── src/
│   ├── Controller/  ← un contrôleur par fonctionnalité
│   ├── Entity/      ← classes métier (Utilisateur, Menu, Commande...)
│   ├── Repository/  ← accès BDD via PDO
│   ├── Service/     ← logique métier (Auth, Mail, Price, Mongo)
│   └── Core/        ← Router, Database, Session
├── templates/       ← vues PHP
├── docker/          ← Dockerfiles et config Nginx
├── sql/             ← create.sql et seed.sql
├── tests/           ← tests PHPUnit
├── docs/            ← documentation
├── .env.example
├── docker-compose.yml
└── docker-compose.prod.yml
```

Lancer l'environnement de développement

```
# 1. Cloner le projet
git clone https://github.com/thomas-bruston/viteGourmandV2.git
cd viteGourmandV2

# 2. Créer le fichier d'environnement
cp .env.example .env

# 3. Lancer les conteneurs
docker compose up -d

# 4. Installer les dépendances
docker compose exec app composer install

# 5. Initialiser la base de données
docker compose exec -T mysql mysql -u root -prootpassword < sql/create.sql
docker compose exec -T mysql mysql -u root -prootpassword --default-character-set=utf8mb4
viteGourmand < sql/seed.sql

# 6. Accéder à l'application
http://localhost:8080
```

3. Modèle Conceptuel de Données (MCD)

(Disponible dans les annexes du dossier projet)

Le MCD comporte 15 tables organisées autour des entités métier principales.

Tables principales

utilisateur : stocke les informations des comptes clients, employés et administrateurs. Liée à role par clé étrangère.

role : quatre valeurs : visiteur, utilisateur, employe, administrateur. La hiérarchie des droits est gérée dans le Router.

menu : contient les informations de chaque menu : titre, description, prix par personne, nombre de personnes minimum, quantité restante. Lié à theme et regime via les tables de liaison menu_theme et menu_regime.

plat : entrée, plat principal ou dessert, lié à un menu via menu_plat. Contient une image en BLOB. Lié aux allergènes via plat_allergene.

commande : table centrale du système métier. Contient toutes les informations de livraison, le statut, les prix calculés, et les flags matériel.

suivi_commande : historique des changements de statut d'une commande avec horodatage.

avis : note (1 à 5) et commentaire, lié à un utilisateur et une commande. Soumis à validation employé via le champ statut.

horaire : horaires d'ouverture modifiables par les employés, affichés dans le footer.

contact : messages envoyés via le formulaire de contact.

Règles d'intégrité

Toutes les clés étrangères sont déclarées avec ON UPDATE CASCADE. Les colonnes email des utilisateurs sont déclarées UNIQUE.

4. Diagramme de classes

(Disponible dans les annexes du dossier projet)

Les entités PHP reflètent directement la structure de la base de données. Chaque entité encapsule ses propriétés en private et n'expose ses données qu'à travers des getters et setters. Les règles métier sont encapsulées directement dans les entités.

5. Diagramme d'utilisation

(Disponible dans les annexes du dossier projet)

L'application expose quatre acteurs aux droits cumulatifs : Visiteur, Client, Employé, Administrateur. Le visiteur accède aux fonctionnalités publiques. Le client hérite du visiteur et accède aux fonctionnalités de commande. L'employé accède à la gestion interne. L'administrateur hérite de l'employé et accède en plus aux statistiques et à la gestion des comptes employés.

6. Diagramme de séquences

(Disponible dans les annexes du dossier projet)

Le diagramme illustre le parcours de passage d'une commande, de la consultation du menu à l'envoi de l'email de confirmation.

7. Documentation du déploiement

Architecture Docker

L'application est conteneurisée via Docker Compose. Quatre conteneurs composent l'architecture :

Conteneur	Image	Rôle
vg_app	PHP 8.3 custom	Exécution du code PHP
vg_nginx	nginx:alpine	Serveur web, reverse proxy
vg_mysql	mysql:8.0	Base de données relationnelle
vg_mongo	mongo:7.0	Base de données non relationnelle

Sécurité du déploiement

Toutes les données sensibles (mots de passe BDD, identifiants MongoDB, clés SMTP) sont externalisées dans un fichier .env non commité dans Git. Le fichier .env.example fourni dans le dépôt documente les variables nécessaires sans exposer les valeurs réelles. En production, HTTPS est activé via la configuration Nginx.

Procédure de déploiement sur VPS

 # 1. Cloner le projet

git clone https://github.com/thomas-bruston/viteGourmandV2.git cd viteGourmandV2

 # 2. Configurer les variables d'environnement

cp .env.example .env

Éditer .env : APP_ENV=production, APP_URL=https://vitegourmand.tech

 # 3. Lancer les conteneurs en production

docker compose -f docker-compose.prod.yml up -d --build

 # 4. Installer les dépendances

docker compose exec app composer install --no-dev --optimize-autoloader

 # 5. Initialiser la base de données

docker compose exec -T mysql mysql -u root -prootpassword < sql/create.sql

docker compose exec -T mysql mysql -u root -prootpassword --default-character-set=utf8mb4 viteGourmand < sql/seed.sql

 # 6. Configurer HTTPS avec Let's Encrypt

sudo apt install certbot -y

docker compose -f docker-compose.prod.yml stop nginx

sudo certbot certonly --standalone -d vitegourmand.tech -d www.vitegourmand.tech

sudo cp /etc/letsencrypt/live/vitegourmand.tech/fullchain.pem docker/nginx/ssl/

sudo cp /etc/letsencrypt/live/vitegourmand.tech/privkey.pem docker/nginx/ssl/

docker compose -f docker-compose.prod.yml start nginx

 # 7. Vérifier que tout fonctionne

docker compose -f docker-compose.prod.yml ps

https://vitegourmand.tech